Token-Bound Accounts for Wildlife: Giving Animals Their Own Wallets

Antje Worring & Zach Kelling

Zoo Labs Foundation

February 2025

Abstract

We present the Zoo implementation of token-bound accounts (TBAs) for wildlife NFTs, based on ERC-6551 and the Lux LRC-6551 standard (LP-9551). Every ZRC-721 wildlife NFT receives a deterministic smart contract wallet that can hold ZRC-20 tokens, other NFTs, and ZRC-1155 badges. This enables wildlife NFTs to receive direct donations for specific animals, accumulate conservation action histories, hold research data artifacts, and participate in Zoo DAO governance through delegated voting power. We describe the TBA registry, account implementation, donation flow, conservation history recording, governance delegation, and cross-chain TBA synchronization. The mechanism directly realizes the "piggybank" concept from the Zoo AGI architecture: each digital animal has a dedicated account that grows in value as the community invests in its protection.

Keywords: token-bound accounts, ERC-6551, wildlife NFT, donations, conservation history, piggybank

1 Introduction

Consider Tiger #42, a tracked Bengal tiger in the Sundarbans represented by a ZRC-721 NFT. Today, a donor who wants to support this specific animal must donate to a general conservation fund and hope the funds are allocated correctly. There is no mechanism to send tokens directly to the tiger, no way for the tiger's NFT to accumulate a conservation history, and no way for the tiger to "own" research data about itself.

Token-bound accounts solve this by giving each NFT its own Ethereum-compatible smart contract wallet. When a ZRC-721 wildlife NFT is minted, a deterministic account is deployed via CREATE2. This account can:

1. Receive $ZOO token donations from anyone.
2. Hold ZRC-20 tokens, ZRC-721 NFTs, and ZRC-1155 badges.
3. Record conservation actions (veterinary care, habitat protection, anti-poaching patrols).
4. Delegate voting power to the NFT owner, giving the animal a "voice" in governance.
5. Transfer its entire contents automatically when the NFT changes hands.

This is the "piggybank" concept from the Zoo AGI paper [4]: each digital animal has a dedicated account that grows in value and information as the community invests in its protection.

1.1 Contributions

1. The IZooTBARegistry and IZooTBA interfaces with conservation extensions (Section 3).
2. The donation flow enabling direct per-animal funding (Section 6).
3. Conservation history recording as an append-only on-chain log (Section 7).
4. Governance delegation from TBA-held tokens (Section 8).
5. Cross-chain TBA synchronization via the Zoo-Lux bridge (Section 9).
6. Security analysis of ownership confusion, circular ownership, and fund drain attacks (Section 11).

2 Background

2.1 ERC-6551

ERC-6551 [1] defines a registry for creating token-bound accounts. The registry uses CREATE2 to deploy a deterministic account for any NFT, given an implementation address, salt, chain ID, token contract, and token ID. The account's `owner()` returns the current owner of the bound NFT, so ownership changes automatically when the NFT transfers.

2.2 The Piggybank Concept

The Zoo AGI architecture [4] proposes that each AI agent and digital animal should have a "piggybank"—a dedicated account that accumulates value from donations, service fees, and yield. TBAs are the on-chain realization of this concept. Unlike a traditional piggybank that merely stores value, a TBA is a full smart contract wallet capable of executing arbitrary transactions, holding diverse asset types, and participating in governance.

2.3 Conservation Fund Routing

The Zoo ecosystem routes conservation funds through verified addresses in the Zoo Contract Registry (ZIP-0100). A TBA's `withdrawToConservation` function restricts withdrawals to registered conservation projects, ensuring that donations to a specific animal actually reach conservation outcomes.

3 Registry Interface

```
interface IZooTBARegistry {
    function createAccount(
        address implementation,
        bytes32 salt,
        uint256 chainId,
        address tokenContract,
        uint256 tokenId
    ) external returns (address account);

    function account(
        address implementation,
        bytes32 salt,
        uint256 chainId,
        address tokenContract,
        uint256 tokenId
    ) external view returns (address);

    event AccountCreated(
        address account,
        address indexed implementation,
        bytes32 salt,
        uint256 chainId,
        address indexed tokenContract,
        uint256 indexed tokenId
    );
}
```

Listing 1: IZooTBARegistry interface

The registry is a singleton deployed at a deterministic address on Zoo L2 (Chain ID: 200200). All TBA addresses are computable before deployment via the standard CREATE2 formula:

$$\text{addr} = \text{keccak256}(\texttt{0xff} \| \text{registry} \| \text{salt}' \| \text{keccak256}(\text{initcode})) \tag{1}$$

where $\text{salt}' = \text{keccak256}(\text{salt} \| \text{chainId} \| \text{tokenContract} \| \text{tokenId})$.

4 Account Implementation

```
interface IZooTBA is IERC165,
    IERC721Receiver, IERC1155Receiver {

    // ERC-6551 Core
    function token() external view returns (
        uint256 chainId,
        address tokenContract,
        uint256 tokenId
    );
    function owner()
        external view returns (address);
    function isValidSigner(
        address signer, bytes calldata context
    ) external view returns (bytes4);
    function state()
        external view returns (uint256);
    function execute(
        address to, uint256 value,
        bytes calldata data, uint8 operation
    ) external payable returns (bytes memory);

    // Conservation Extensions
    function totalDonationsReceived()
        external view returns (uint256);
    function donorCount()
        external view returns (uint256);
    function donationFrom(address donor)
        external view returns (uint256);
    function donate(uint256 amount) external;
    function withdrawToConservation(
        address fund, uint256 amount
    ) external;

    event DonationReceived(
        address indexed donor,
        uint256 amount,
        uint256 totalDonations
    );
    event ConservationWithdrawal(
        address indexed fund,
        uint256 amount,
        address indexed authorizedBy
    );

    // Conservation History
    struct ConservationAction {
        uint256 timestamp;
        address initiator;
```

```
        string actionType;  // "donation",
            // "observation", "medical",
            // "relocation", "research"
        uint256 value;      // ZOO amount or 0
        string detailsCID;  // IPFS CID
    }

    function conservationHistory()
        external view returns (
            ConservationAction[] memory);
    function recordConservationAction(
        string calldata actionType,
        uint256 value,
        string calldata detailsCID
    ) external;

    event ConservationActionRecorded(
        string actionType,
        address indexed initiator,
        uint256 value,
        uint256 timestamp
    );

    // Governance Delegation
    function delegateVotingPower(
        address token, address delegatee
    ) external;
}
```

Listing 2: IZooTBA interface

5 Automatic TBA Provisioning

ZRC-721 contracts conforming to ZIP-0701 automatically create a TBA at mint time:

```
function _afterTokenTransfer(
    address from, address to,
    uint256 tokenId, uint256 batchSize
) internal override {
    super._afterTokenTransfer(
        from, to, tokenId, batchSize);
    if (from == address(0)) {
        // Mint: create TBA
        IZooTBARegistry(ZOO_TBA_REGISTRY)
            .createAccount(
                ZOO_TBA_IMPLEMENTATION,
                bytes32(0),
                block.chainid,
                address(this),
                tokenId
            );
    }
}
```

Listing 3: Automatic TBA creation on mint

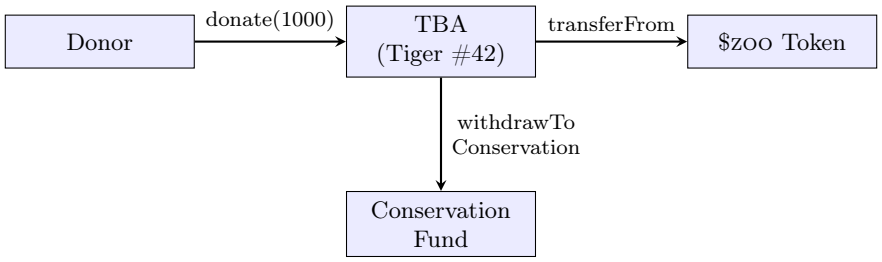

Figure 1: Donation flow: a donor sends $ZOO directly to a wildlife NFT's TBA. The NFT owner later withdraws to a verified conservation fund.

This ensures every wildlife NFT has a functional account from the moment of creation. Donors can send $ZOO to the computed TBA address even before the TBA contract is deployed; the funds will be accessible once the account is created.

6 Donation Flow

1. A donor calls `donate(1000)` on Tiger #42's TBA.
2. The TBA transfers 1,000 $ZOO from the donor to itself via `transferFrom`.
3. The TBA records the donation: updates `totalDonationsReceived`, increments `donorCount` (if new donor), and records the per-donor amount.
4. A `DonationReceived` event is emitted.
5. A conservation action record is appended: `actionType: "donation"`.
6. Later, the NFT owner calls `withdrawToConservation(fund, 500)`.
7. The TBA verifies the fund address against the Zoo Contract Registry (ZIP-0100).
8. 500 $ZOO is transferred to the conservation fund.
9. A `ConservationWithdrawal` event is emitted.

Theorem 6.1 (Donation Integrity)**.** *All $ZOO tokens held by a TBA are traceable to specific donors. The `donationFrom` mapping provides a complete accounting of all inflows, and the conservation history provides a complete accounting of all outflows.*

7 Conservation History

Each TBA maintains an append-only conservation action log. Action types include:

Records are written by:

- The NFT owner (for any action type).
- Authorized verifiers in the Zoo Contract Registry (for "verified" records that carry higher trust).
- The TBA itself (automatically, for donations and withdrawals).

Action Type	Value Field	Details
donation	ZOO amount	Donor address, message
observation	0	GPS coordinates, photo CID
medical	ZOO cost	Procedure, veterinarian
relocation	0	Origin, destination, reason
research	0	Dataset CID, researcher

Table 1: Conservation action types recorded in TBA history.

This on-chain history enables donors to verify that their contributions to Tiger #42 resulted in actual conservation outcomes: veterinary care, anti-poaching patrols, habitat monitoring.

8 Governance Delegation

Wildlife NFTs that hold $ZOO tokens through their TBA can participate in Zoo DAO governance:

```
function delegateVotingPower(
    address token, address delegatee
) external {
    require(msg.sender == owner(),
        "only NFT owner");
    IZRC20Votes(token).delegate(delegatee);
}
```

Listing 4: Governance delegation from TBA

The NFT owner delegates the TBA's $ZOO voting power to a chosen delegatee (typically themselves). This means:

- Tiger #42's $ZOO donations contribute to governance voting power.
- The more a community donates to an animal, the more governance influence that animal carries.
- The human owner votes "on behalf of" the animal, creating a symbolic "voice for nature."

Definition 8.1 (Wildlife Governance Weight)**.** *For a wildlife NFT with TBA holding B $ZOO tokens, its governance weight is:*

$$W_{wildlife} = B \cdot (1 + \beta \cdot R_{species}) \tag{2}$$

where $R_{species}$ is a multiplier based on IUCN threat status: CR = 2.0, EN = 1.5, VU = 1.0, NT = 0.5, LC = 0.25.

This weights governance influence toward more endangered species, aligning governance power with conservation urgency.

9 Cross-Chain TBA Synchronization

When a ZRC-721 NFT is bridged to Lux C-Chain via ZIP-0800:

1. The TBA on Zoo L2 is frozen: no new `execute` calls are accepted.
2. A mirror TBA is created on Lux C-Chain using the same salt and implementation.
3. Held assets remain on Zoo L2. They are accessible via the Zoo-Lux bridge.
4. When the NFT returns to Zoo L2, the original TBA is unfrozen and the Lux mirror is deactivated.

The freeze is enforced by incrementing the TBA's `state()` nonce on bridge-out. Any pending operations signed with the old nonce are invalidated.

Theorem 9.1 (Cross-Chain Asset Safety)**.** *A TBA's assets cannot be double-spent across chains. The freeze mechanism ensures exactly one chain has an active TBA at any time. Asset access on the inactive chain requires a bridge transfer, which is mediated by the MPC signer set (ZIP-0800).*

10 Reference Implementation

```
contract ZooTBA is IZooTBA, ERC165 {
    uint256 private _state;
    mapping(address => uint256) private _donations;
    uint256 private _totalDonations;
    uint256 private _donorCount;
    ConservationAction[] private _history;

    function donate(uint256 amount) external {
        IERC20(ZOO_TOKEN).transferFrom(
            msg.sender, address(this), amount);
        if (_donations[msg.sender] == 0)
            _donorCount++;
        _donations[msg.sender] += amount;
        _totalDonations += amount;
        _history.push(ConservationAction({
            timestamp: block.timestamp,
            initiator: msg.sender,
            actionType: "donation",
            value: amount,
            detailsCID: ""
        }));
        emit DonationReceived(
            msg.sender, amount, _totalDonations);
    }

    function withdrawToConservation(
        address fund, uint256 amount
    ) external {
        require(msg.sender == owner(),
            "only NFT owner");
        require(IRegistry(ZOO_REGISTRY)
            .isConservationFund(fund),
            "unverified fund");
        IERC20(ZOO_TOKEN).transfer(fund, amount);
        _history.push(ConservationAction({
            timestamp: block.timestamp,
            initiator: msg.sender,
            actionType: "withdrawal",
```

```
            value: amount,
            detailsCID: ""
        }));
        emit ConservationWithdrawal(
            fund, amount, msg.sender);
    }

    function owner() public view returns (address) {
        (,address tokenContract, uint256 tokenId)
            = token();
        return IERC721(tokenContract)
            .ownerOf(tokenId);
    }

    function execute(
        address to, uint256 value,
        bytes calldata data, uint8 operation
    ) external payable returns (bytes memory) {
        require(msg.sender == owner(),
            "only NFT owner");
        require(operation == 0, "only call");
        _state++;
        (bool success, bytes memory result) =
            to.call{value: value}(data);
        require(success, "execution failed");
        return result;
    }
}
```

Listing 5: ZooTBA reference implementation (simplified)

11 Security Analysis

11.1 Ownership Confusion

The TBA's `owner()` changes when the NFT transfers. The `state()` nonce increments on each ownership change, invalidating pending operations signed by the previous owner.

11.2 Reentrancy

The `execute` function can call arbitrary contracts. The implementation increments `_state` before the external call (checks-effects-interactions) and uses a reentrancy guard.

11.3 Fund Drain

A compromised NFT owner key could drain the TBA. Mitigations:

- `withdrawToConservation` only sends to verified conservation funds.
- The general `execute` function is subject to rate limiting (optional extension).
- Multi-sig requirements can be configured for withdrawals exceeding a threshold.

11.4 Circular Ownership

A TBA could hold its own parent NFT, creating a circular ownership loop where the TBA owns the NFT that owns the TBA. Implementations MUST prevent an account from acquiring its own bound token.

11.5 Conservation Fund Verification

The `withdrawToConservation` function verifies the destination against the live Zoo Contract Registry. A stale local cache could allow withdrawals to deregistered addresses. The implementation queries the registry on every call.

11.6 Gas Griefing

Unbounded conservation history arrays could make `conservationHistory()` calls exceed gas limits. Implementations SHOULD paginate history queries and set a per-account record limit (e.g., 10,000 records).

12 Related Work

ERC-6551 [1] defines the token-bound account standard. Tokenbound [2] provides SDK and infrastructure for ERC-6551 accounts. Lens Protocol [12] uses NFT-based profiles with associated data; TBAs generalize this to any NFT. Bored Ape Yacht Club's "summoning" mechanic [13] gives NFTs agent-like qualities; TBAs formalize the underlying account model.

In conservation finance, the GiveDirectly [14] model of direct cash transfers to individuals shares conceptual DNA with TBA-based per-animal donations: both remove intermediaries between donors and beneficiaries.

The zoo-agi paper [4] introduced the piggybank concept for AI agents. This paper realizes the same concept for wildlife NFTs, extending it with conservation-specific features (verified fund withdrawals, action history, species-weighted governance).

13 Conclusion

Token-bound accounts transform static wildlife NFTs into living on-chain entities that accumulate value, history, and governance influence. The mechanism creates a direct link between a donor and a specific animal, habitat, or research program—no intermediary fund management required. When the community donates to Tiger #42, those tokens stay with Tiger #42, grow Tiger #42's governance voice, and are withdrawable only to verified conservation projects.

This is the on-chain realization of a simple idea: every animal deserves its own wallet.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the Tokenbound team for the ERC-6551 reference implementation and the Lux team for the LRC-6551 standard.

References

[1] J. Fang et al., "EIP-6551: Non-fungible Token Bound Accounts," Ethereum Improvement Proposals, 2023.

[2] Tokenbound, "Tokenbound: Infrastructure for ERC-6551," `https://tokenbound.org`, 2023.

[3] Lux Partners, "LP-9551: LRC-6551 Token-Bound Accounts," Lux Protocol Standards, 2023.

[4] Z. Kelling et al., "Zoo AGI: Autonomous Agents for Conservation," Zoo Labs Foundation Technical Report, 2024.

[5] W. Entriken et al., "EIP-721: Non-Fungible Token Standard," Ethereum Improvement Proposals, 2018.

[6] Zoo Labs Foundation, "ZIP-0100: Zoo Contract Registry," Zoo Improvement Proposals, 2024.

[7] Zoo Labs Foundation, "ZIP-0500: ESG Principles for Conservation Impact," Zoo Improvement Proposals, 2024.

[8] Zoo Labs Foundation, "ZIP-0700: ZRC-20 Fungible Token Standard," Zoo Improvement Proposals, 2025.

[9] Zoo Labs Foundation, "ZIP-0701: ZRC-721 NFT Standard," Zoo Improvement Proposals, 2025.

[10] Zoo Labs Foundation, "ZIP-0800: Zoo-Lux Bridge Protocol," Zoo Improvement Proposals, 2025.

[11] V. Buterin et al., "EIP-4337: Account Abstraction Using Alt Mempool," Ethereum Improvement Proposals, 2021.

[12] Lens Protocol, "Lens: Decentralized Social Graph," Technical Documentation, 2022.

[13] Yuga Labs, "Bored Ape Yacht Club: NFT Collection with Utility," 2022.

[14] GiveDirectly, "Direct Cash Transfers to the Extreme Poor," `https://givedirectly.org`, 2020.

[15] Safe, "Safe: Multi-Signature Wallet Infrastructure," Technical Documentation, 2021.

[16] W. Radomski et al., "EIP-1155: Multi Token Standard," Ethereum Improvement Proposals, 2018.

[17] OpenZeppelin, "OpenZeppelin Contracts," `https://openzeppelin.com/contracts`, 2023.

[18] Zoo Labs Foundation, "ZIP-0015: Zoo L2 Chain Architecture," Zoo Improvement Proposals, 2024.

[19] V. Buterin, "EIP-1014: Skinny CREATE2," Ethereum Improvement Proposals, 2018.

[20] C. Reitwießner et al., "EIP-165: Standard Interface Detection," Ethereum Improvement Proposals, 2018.